Mathematics for AI/ML

Linear algebra, calculus, probability, statistics, optimisation and information theory - the minimum maths needed to truly understand ML.

Contents

#	Topic	Level
1	Scalars, Vectors, Matrices, Tensors	Beginner
2	Vector operations: dot product, norm, cosine similarity	Beginner
3	Matrix operations: multiplication, transpose, inverse, determinant	Beginner
4	Systems of linear equations	Beginner
5	Eigenvalues and Eigenvectors	Intermediate
6	SVD (Singular Value Decomposition)	Intermediate
7	Vector spaces, basis, rank, projection, orthogonality	Intermediate
8	Limits, derivatives and differentiation rules	Beginner
9	Partial derivatives, gradient, Jacobian, Hessian	Intermediate
10	Chain rule and computational graphs	Intermediate
11	Integrals (basics)	Beginner
12	Gradient descent and optimisation basics	Intermediate
13	Convex vs non-convex optimisation, local minima, saddle points	Advanced
14	Lagrange multipliers and constrained optimisation	Advanced
15	Probability basics: events, conditional, independence	Beginner
16	Bayes' theorem	Beginner
17	Random variables, PMF, PDF, CDF	Beginner
18	Common distributions: Bernoulli, Binomial, Poisson, Uniform, Normal, Exponential	Beginner
19	Joint, marginal and conditional distributions; covariance and correlation	Intermediate
20	Law of large numbers and Central Limit Theorem	Intermediate
21	Descriptive statistics: mean, median, mode, variance, std, skew, kurtosis, percentiles	Beginner
22	Sampling, estimation, Maximum Likelihood (MLE), MAP	Intermediate
23	Confidence intervals and hypothesis testing (z, t, chi-square, ANOVA, p-value)	Intermediate
24	A/B testing and statistical power	Intermediate
25	Bayesian statistics, priors, posteriors, conjugacy	Advanced
26	Information theory: entropy, cross-entropy, KL divergence, mutual information	Intermediate
27	Markov chains and stochastic processes	Advanced
28	Discrete maths, graph theory, combinatorics (essentials)	Intermediate
29	Numerical computing: floating point, stability, log-sum-exp	Advanced

1. Scalars, Vectors, Matrices, Tensors [Beginner]

Explanation (English)

A scalar is a single number, a vector is a 1-D list of numbers, a matrix is 2-D, a tensor is N-D. Data points are vectors; datasets are matrices; images/batches are tensors.

Tanglish (Tamil + English) Vilakkam

Scalar = oru number. Vector = number-oda list. Matrix = rows & columns table. Tensor = N-dimension array. ML-la oru data point vector, dataset matrix, image batch tensor.

Example / Program

```
import numpy as np
s=5
v=np.array([1,2,3])
M=np.array([[1,2],[3,4]])
T=np.zeros((2,3,4))
print(v.shape,M.shape,T.shape,T.ndim)
```

Learn more

YouTube search: Scalars, Vectors, Matrices, Tensors tutorial

2. Vector operations: dot product, norm, cosine similarity [Beginner]

Explanation (English)

Dot product a.b = sum(ai*bi) measures alignment. L2 norm is the length; cosine similarity = a.b/(|a||b|) is used in embeddings and search.

Tanglish (Tamil + English) Vilakkam

Dot product rendu vector evlo orae direction-la irukku-nu kaattum. Norm = length. Cosine similarity text/embedding similarity-ku use aagum.

Example / Program

```
import numpy as np
a=np.array([1,2,3]);b=np.array([4,5,6])
print(a@b, np.linalg.norm(a))
print(a@b/(np.linalg.norm(a)*np.linalg.norm(b)))
```

Learn more

YouTube search: Vector operations: dot product, norm, cosine similarity tutorial

3. Matrix operations: multiplication, transpose, inverse, determinant [Beginner]

Explanation (English)

Matrix multiplication composes linear transformations; shapes must match (m,n)x(n,p). Inverse exists only for square non-singular matrices (det != 0).

Tanglish (Tamil + English) Vilakkam

Matrix multiply pannumbodhu (m,n)x(n,p) shape match aaganum. Inverse square matrix-ku mattum, determinant 0 illama irundha mattum.

Example / Program

```
import numpy as np
A=np.array([[2,1],[1,3]]);B=np.eye(2)
print(A@B, A.T, np.linalg.inv(A), np.linalg.det(A))
```

Learn more

YouTube search: Matrix operations: multiplication, transpose, inverse, determinant tutorial

4. Systems of linear equations [Beginner]

Explanation (English)

Ax=b can have one, none or infinite solutions. Solved by Gaussian elimination / np.linalg.solve; linear regression is solving an over-determined system in the least-squares sense.

Tanglish (Tamil + English) Vilakkam

Ax=b equation-a solve pannalam. Linear regression-um oru least squares system solve panradhu dhaan.

Example / Program

```
import numpy as np
A=np.array([[2,1],[1,3]]);b=np.array([3,5])
print(np.linalg.solve(A,b))
```

Learn more

YouTube search: Systems of linear equations tutorial

5. Eigenvalues and Eigenvectors [Intermediate]

Explanation (English)

For Av = lambda v, v is an eigenvector (direction unchanged by A) and lambda its eigenvalue (scaling). Used in PCA, spectral clustering, PageRank, stability analysis.

Tanglish (Tamil + English) Vilakkam

Av = lambda v. Eigenvector direction maaradhu, eigenvalue scale mattum maarum. PCA, PageRank-la use aagum.

Example / Program

```
import numpy as np
A=np.array([[2,0],[0,3]])
w,v=np.linalg.eig(A);print(w,v)
```

Learn more

YouTube search: Eigenvalues and Eigenvectors tutorial

6. SVD (Singular Value Decomposition) [Intermediate]

Explanation (English)

Any matrix A = U S V^T. Truncated SVD gives best low-rank approximation; used in PCA, recommender systems, image compression, LSA.

Tanglish (Tamil + English) Vilakkam

SVD matrix-a U, S, V-nu pirikkum. Dimensionality reduction, recommender, image compression-la use.

Example / Program

```
import numpy as np
A=np.random.rand(5,3)
U,S,Vt=np.linalg.svd(A,full_matrices=False)
print(S)
```

Learn more

YouTube search: SVD (Singular Value Decomposition) tutorial

7. Vector spaces, basis, rank, projection, orthogonality [Intermediate]

Explanation (English)

A basis is a minimal set of vectors spanning a space; rank = number of independent columns; projection finds the closest point in a subspace (basis of least squares).

Tanglish (Tamil + English) Vilakkam

Basis = space-a span panra minimum vectors. Rank = independent columns count. Projection = nearest point in subspace.

Example / Program

```
import numpy as np
A=np.array([[1,2],[2,4]])
print(np.linalg.matrix_rank(A))
```

Learn more

YouTube search: Vector spaces, basis, rank, projection, orthogonality tutorial

8. Limits, derivatives and differentiation rules [Beginner]

Explanation (English)

The derivative f'(x) is the slope / rate of change. Rules: power, product, quotient, chain. Neural nets rely on the chain rule (backprop).

Tanglish (Tamil + English) Vilakkam

Derivative = slope / maatram rate. Chain rule dhaan backpropagation-oda base.

Example / Program

```
import sympy as sp
x=sp.symbols('x')
print(sp.diff(x**3+2*x,x), sp.diff(sp.sin(x**2),x))
```

Learn more

YouTube search: Limits, derivatives and differentiation rules tutorial

9. Partial derivatives, gradient, Jacobian, Hessian [Intermediate]

Explanation (English)

Gradient = vector of partial derivatives (direction of steepest ascent). Jacobian = matrix of first derivatives of a vector function; Hessian = matrix of second derivatives (curvature).

Tanglish (Tamil + English) Vilakkam

Gradient = ella partial derivatives-oda vector, steepest ascent direction. Hessian = curvature.

Example / Program

```
import sympy as sp
x,y=sp.symbols('x y')
f=x**2*y+y**3
print([sp.diff(f,v) for v in (x,y)])
```

Learn more

YouTube search: Partial derivatives, gradient, Jacobian, Hessian tutorial

10. Chain rule and computational graphs [Intermediate]

Explanation (English)

dz/dx = dz/dy * dy/dx. Frameworks build a computational graph and apply the chain rule backwards (autograd).

Tanglish (Tamil + English) Vilakkam

Chain rule-a graph-la reverse-la apply panradhu dhaan autograd / backprop.

Example / Program

```
import torch
x=torch.tensor(2.,requires_grad=True)
y=(x**2+1)**3
y.backward();print(x.grad)
```

Learn more

YouTube search: Chain rule and computational graphs tutorial

11. Integrals (basics) [Beginner]

Explanation (English)

Integral = area under a curve / accumulated sum. Needed for probability densities (area=1), expected values and continuous distributions.

Tanglish (Tamil + English) Vilakkam

Integral = curve keezha area. Probability density, expectation calculate panna venum.

Example / Program

```
from scipy.integrate import quad
import numpy as np
print(quad(lambda x: np.exp(-x**2),-np.inf,np.inf))
```

Learn more

YouTube search: Integrals (basics) tutorial

12. Gradient descent and optimisation basics [Intermediate]

Explanation (English)

Update parameters opposite to the gradient: w = w - lr * dL/dw. Learning rate too high diverges, too low is slow. Variants: batch, stochastic, mini-batch, momentum, Adam.

Tanglish (Tamil + English) Vilakkam

Gradient-oda opposite direction-la weight update pannuvom. Learning rate romba perusa irundha diverge, romba sinna-na slow.

Example / Program

```
w=5.0;lr=0.1
for i in range(50):
    g=2*(w-3)   # f=(w-3)^2
    w-=lr*g
print(w)  # ~3
```

Learn more

YouTube search: Gradient descent and optimisation basics tutorial

13. Convex vs non-convex optimisation, local minima, saddle points [Advanced]

Explanation (English)

Convex losses have one global minimum (linear/logistic regression). Deep nets are non-convex with saddle points; SGD noise, momentum and good initialisation help.

Tanglish (Tamil + English) Vilakkam

Convex-la oru global minimum. Deep learning non-convex - local minima, saddle points irukku.

Learn more

YouTube search: convex vs non convex optimization machine learning

14. Lagrange multipliers and constrained optimisation [Advanced]

Explanation (English)

Optimise f(x) subject to g(x)=0 using L = f + lambda*g. Underlies SVM (dual form) and many regularisation interpretations.

Tanglish (Tamil + English) Vilakkam

Constraint irukkum bodhu optimise panna Lagrange multiplier. SVM-la use aagum.

Learn more

YouTube search: lagrange multipliers explained

15. Probability basics: events, conditional, independence [Beginner]

Explanation (English)

P(A) in [0,1]. Conditional P(A|B)=P(A and B)/P(B). Independent events: P(A and B)=P(A)P(B). Sum and product rules.

Tanglish (Tamil + English) Vilakkam

Probability 0 to 1. Conditional probability = B nadandha apram A nadakkura chance.

Example / Program

```
from itertools import product
r=list(product(range(1,7),repeat=2))
print(sum(1 for a,b in r if a+b==7)/len(r))
```

Learn more

YouTube search: Probability basics: events, conditional, independence tutorial

16. Bayes' theorem [Beginner]

Explanation (English)

P(A|B) = P(B|A)P(A)/P(B). Foundation of Naive Bayes, Bayesian inference and spam filters.

Tanglish (Tamil + English) Vilakkam

Bayes theorem: prior + evidence = posterior. Spam filter, Naive Bayes-la use.

Example / Program

```
pA=0.01;sens=0.9;fp=0.05
pos=sens*pA+fp*(1-pA)
```

```
print(sens*pA/pos)
```

Learn more
YouTube search: Bayes' theorem tutorial

17. Random variables, PMF, PDF, CDF [Beginner]

Explanation (English)
Discrete variables have a PMF, continuous ones a PDF; the CDF gives P(X<=x). Expectation E[X] and variance Var[X] summarise them.

Tanglish (Tamil + English) Vilakkam
Discrete-ku PMF, continuous-ku PDF, CDF = x varaikkum probability. E[X] = average, Var = spread.

Example / Program
```
from scipy import stats
print(stats.norm.pdf(0),stats.norm.cdf(1.96))
```

Learn more
YouTube search: Random variables, PMF, PDF, CDF tutorial

18. Common distributions: Bernoulli, Binomial, Poisson, Uniform, Normal, Exponential [Beginner]

Explanation (English)
Bernoulli: one yes/no trial; Binomial: n trials; Poisson: events per interval; Uniform: equal chance; Normal: bell curve (CLT); Exponential: waiting time.

Tanglish (Tamil + English) Vilakkam
Normal = bell curve. Binomial = n trials-la ethana success. Poisson = oru nerathula ethana events.

Example / Program
```
import numpy as np
print(np.random.binomial(10,.5,5),np.random.poisson(3,5),np.random.normal(0,1,3))
```

Learn more
YouTube search: Common distributions: Bernoulli, Binomial, Poisson, Uniform, Normal, Exponential tutorial

19. Joint, marginal and conditional distributions; covariance and correlation [Intermediate]

Explanation (English)
Joint P(X,Y); marginalise by summing out one variable. Covariance measures co-movement; correlation is scale-free in [-1,1].

Tanglish (Tamil + English) Vilakkam
Covariance/correlation rendu variable eppadi serndhu maaruthu-nu sollum.

Example / Program
```
import numpy as np
x=np.random.rand(100);y=2*x+np.random.rand(100)*.1
print(np.corrcoef(x,y)[0,1])
```

Learn more
YouTube search: Joint, marginal and conditional distributions; covariance and correlation tutorial

20. Law of large numbers and Central Limit Theorem [Intermediate]

Explanation (English)
Sample mean converges to the true mean (LLN); the distribution of sample means approaches Normal regardless of the population shape (CLT).

Tanglish (Tamil + English) Vilakkam
Sample size perusa aaga aaga mean true mean-ku pogum (LLN). Sample means normal distribution aagum (CLT).

Example / Program
```
import numpy as np
```

```
m=[np.random.exponential(1,50).mean() for _ in range(2000)]
print(np.mean(m),np.std(m))
```

Learn more
YouTube search: Law of large numbers and Central Limit Theorem tutorial

21. Descriptive statistics: mean, median, mode, variance, std, skew, kurtosis, percentiles [Beginner]

Explanation (English)
Central tendency (mean, median, mode), spread (variance, std, IQR), shape (skewness, kurtosis). Median is robust to outliers.

Tanglish (Tamil + English) Vilakkam
Mean, median, mode = center. Variance, std = spread. Median outlier-ku bayappadaadhu.

Example / Program
```
import pandas as pd
s=pd.Series([1,2,2,3,100]);print(s.describe(),s.skew())
```

Learn more
YouTube search: Descriptive statistics: mean, median, mode, variance, std, skew, kurtosis, percentiles tutorial

22. Sampling, estimation, Maximum Likelihood (MLE), MAP [Intermediate]

Explanation (English)
MLE picks parameters that maximise the probability of the observed data; MAP adds a prior (equals regularisation). Many loss functions are negative log-likelihoods.

Tanglish (Tamil + English) Vilakkam
MLE = data-va adhigama explain panra parameters. MAP = MLE + prior (regularisation).

Example / Program
```
import numpy as np
d=np.random.normal(5,2,1000)
print(d.mean(),d.std())  # MLE of Normal
```

Learn more
YouTube search: Sampling, estimation, Maximum Likelihood (MLE), MAP tutorial

23. Confidence intervals and hypothesis testing (z, t, chi-square, ANOVA, p-value) [Intermediate]

Explanation (English)
Null vs alternative hypothesis; p-value = probability of data at least this extreme if H0 is true. t-test compares means, chi-square tests categorical association, ANOVA compares >2 groups.

Tanglish (Tamil + English) Vilakkam
H0 vs H1. p-value chinnadha irundha H0-a reject pannalam. t-test means compare, chi-square categories.

Example / Program
```
from scipy import stats
import numpy as np
a=np.random.normal(0,1,50);b=np.random.normal(.5,1,50)
print(stats.ttest_ind(a,b))
```

Learn more
YouTube search: Confidence intervals and hypothesis testing (z, t, chi-square, ANOVA, p-value) tutorial

24. A/B testing and statistical power [Intermediate]

Explanation (English)
Compare variants with randomised experiments; plan sample size from effect size, alpha and power. Beware peeking and multiple comparisons.

Tanglish (Tamil + English) Vilakkam
A/B test - rendu version compare. Sample size munnadiye plan pannanum, peek pannakoodadhu.

Example / Program

```
from statsmodels.stats.proportion import proportions_ztest
print(proportions_ztest([60,80],[1000,1000]))
```

Learn more

YouTube search: A/B testing and statistical power tutorial

25. Bayesian statistics, priors, posteriors, conjugacy [Advanced]

Explanation (English)

Update beliefs with data: posterior ∝ likelihood × prior. Conjugate priors (Beta-Binomial) give closed forms; MCMC / variational inference handle the rest.

Tanglish (Tamil + English) Vilakkam

Bayesian approach: prior belief + data = posterior belief. MCMC (PyMC) use pannuvom.

Example / Program

```
from scipy import stats
print(stats.beta(1+7,1+3).mean())  # 7 heads in 10
```

Learn more

YouTube search: Bayesian statistics, priors, posteriors, conjugacy tutorial

26. Information theory: entropy, cross-entropy, KL divergence, mutual information [Intermediate]

Explanation (English)

Entropy H = -sum p log p measures uncertainty. Cross-entropy is the classification loss; KL divergence measures how one distribution differs from another; mutual information measures shared info.

Tanglish (Tamil + English) Vilakkam

Entropy = uncertainty. Cross-entropy classification loss. KL divergence = rendu distribution difference.

Example / Program

```
import numpy as np
p=np.array([.5,.5]);q=np.array([.9,.1])
print(-(p*np.log2(p)).sum(), (p*np.log(p/q)).sum())
```

Learn more

YouTube search: Information theory: entropy, cross-entropy, KL divergence, mutual information tutorial

27. Markov chains and stochastic processes [Advanced]

Explanation (English)

A Markov chain has memoryless transitions between states given by a transition matrix; it has a stationary distribution. Foundation of HMMs, MCMC and RL.

Tanglish (Tamil + English) Vilakkam

Next state current state-a mattum depend aagum. HMM, RL, MCMC-ku base.

Example / Program

```
import numpy as np
P=np.array([[.9,.1],[.5,.5]]);s=np.array([1,0])
for _ in range(50): s=s@P
print(s)
```

Learn more

YouTube search: Markov chains and stochastic processes tutorial

28. Discrete maths, graph theory, combinatorics (essentials) [Intermediate]

Explanation (English)

Permutations, combinations, graphs (nodes/edges, paths, centrality) - needed for algorithms, GNNs, knowledge graphs and recommender systems.

Tanglish (Tamil + English) Vilakkam

Permutation, combination, graph basics. GNN, recommender-ku venum.

Example / Program

```
import math, networkx as nx
print(math.comb(5,2));G=nx.path_graph(4);print(nx.shortest_path(G,0,3))
```

Learn more

YouTube search: Discrete maths, graph theory, combinatorics (essentials) tutorial

29. Numerical computing: floating point, stability, log-sum-exp [Advanced]

Explanation (English)

Floats have limited precision; overflow/underflow in exp/log is avoided with log-sum-exp and stable softmax. Always standardise inputs.

Tanglish (Tamil + English) Vilakkam

Floating point precision limited. Softmax stable-a irukka max subtract pannanum.

Example / Program

```
import numpy as np
x=np.array([1000.,1001.])
print(np.exp(x-x.max())/np.exp(x-x.max()).sum())
```

Learn more

YouTube search: Numerical computing: floating point, stability, log-sum-exp tutorial